

06. 자료 구조와 알고리즘

12. 해시 테이블

학습목표

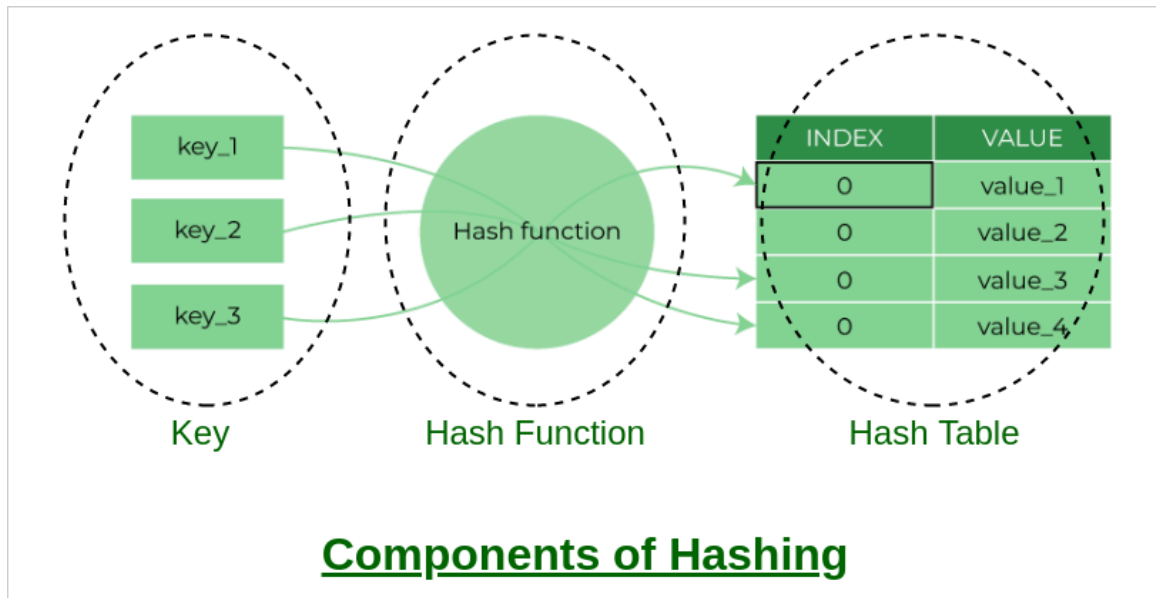
- 해시 테이블에 대해서 이해할 수 있다.
- 해시 테이블을 적재적소에 활용할 수 있다.

들어가며

집합(Set)은 데이터를 중복되지 않게(Unique) 저장하는 추상 자료형이다. 수학에서의 유한 집합(Finite Set)을 컴퓨터에 구현했다고 보면 된다. **맵*(Map)**은 키와 이에 연관되는 값을 함께 저장하는 추상 자료형으로 이때 키는 유일해야 한다. 중복된 데이터(키)를 허용하는 자료형을 **멀티셋(Multiset)**, **멀티맵(Multimap)**이라고 한다.

* 연관 배열(Associative Array), 사전(Dictionary)이라고도 한다.

둘은 저장되는 타입만 다를 뿐 똑같기 때문에 구현하는 방식도 같다. 배열, 이진 검색 트리, 해시 테이블을 사용하게 되는데, 우리는 이미 사용해본 바가 있다. 룩업 테이블이나 DFS, BFS 때 사용했던 방문 여부를 저장하기 위한 배열이 그렇다. 배열과 이진 검색 트리에 대해서는 이미 배웠으므로 이번에는 **해시 테이블(Hash Table)**을 중점적으로 다룬다.



해시 테이블의 모습

해싱

배열은 모든 원소에 $O(1)$ 으로 접근할 수 있다. 인덱스만 알면 주소를 계산해낼 수 있기 때문이다. 그래서 룩업테이블처럼 어떤 데이터를 인덱스로 바꿀 수만 있다면 매우 유용하다. 하지만 모든 데이터가 정수 타입이 아니기 때문에 인덱스로 바꿀 순 없다. 그래서 어떤 타입의 데이터든 인덱스로 바꾸기 위해 해싱을 사용하는 것이 해시 테이블의 핵심이다.

해싱(Hashing)은 어떤 길이의 데이터든 입력의 크기에 상관 없이 일정 크기의 값으로 변환하는 것을 말하며 이런 기능을 하는 함수를 해시 함수(Hash Function)라고 한다. 하지만 위의 정의를 만족한다고 해서 모든 것이 해시 함수인 것은 아니다. 아래의 예시를 보자.

```
// 아래의 함수는 이론대로라면 모든 데이터에 대해서 일정한 데이터로 변환한다.
```

```
int Hash(object o)
{
    return 4;
}
```

위의 예시는 해시 함수라 하지 않는다. 해시 함수는 아래의 성질을 만족해야 한다.

1) 균일성(Uniformity)

균일성은 **충돌이 발생할 확률이 낮은 것**을 의미한다. **충돌(Collision)**이란 **서로 다른 입력에 대해 같은 해시 값이 생성된 것**이다. 해시 값의 범위에 고르게 분포되도록 계산되는 것이 좋다.

2) 효율성(Effectivity)

효율성은 **계산이 쉬운 것**을 의미한다. 아무리 균일성이 높다고 해도 해시 값을 계산하는 데 오래 걸린다면 효율성이 떨어진다.

3) 결정적(Deterministic)

결정적이라는 것은 **같은 입력에 대해 같은 해시 값이 생성되는 것**을 말한다.

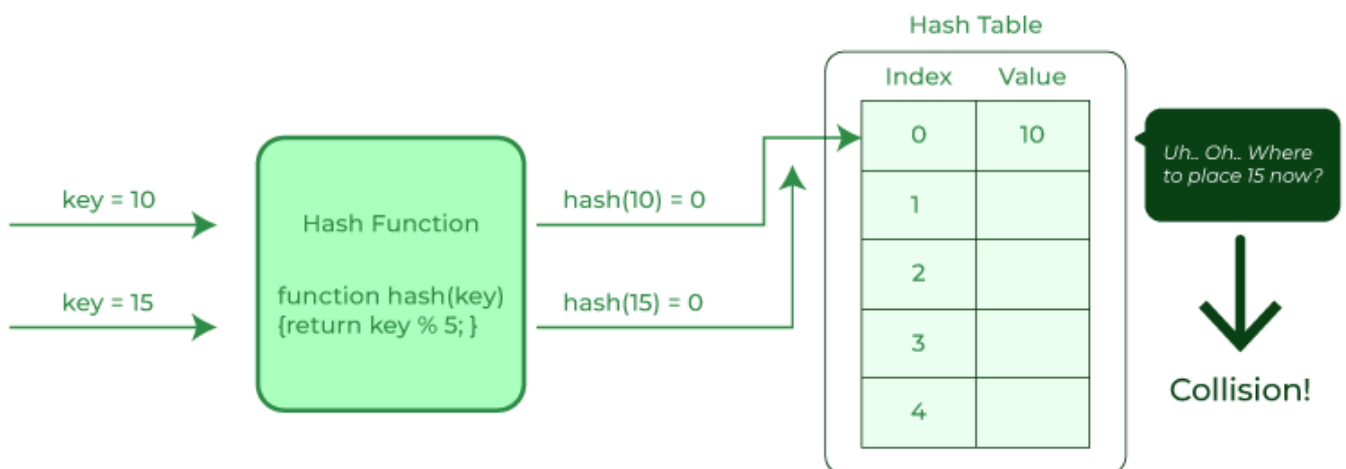
해시 함수의 이런 특징 때문에 해시는 **보안**과 관련된 기술에 많이 사용된다. 디지털 서명, 공인인증서 등 우리 주변에서 흔히 볼 수 있다. 해시 함수를 만드는 방법도 있지만 여기서는 다루지 않는다.* 보통 해시 함수는 이미 구현된 것을 많이 사용한다. 가장 많이 쓰이는 것은 SHA-256, SHA-512 등이다.

* 관심 있는 사람은 [여기](#)를 참고하라.

충돌 처리

해시 테이블에서 충돌은 피할 수 없다. 메모리 용량을 해시 값의 범위만큼 미리 확보해둘 수 없기 때문이다. 충돌을 처리하는 방법에는 2가지 방법이 있다.

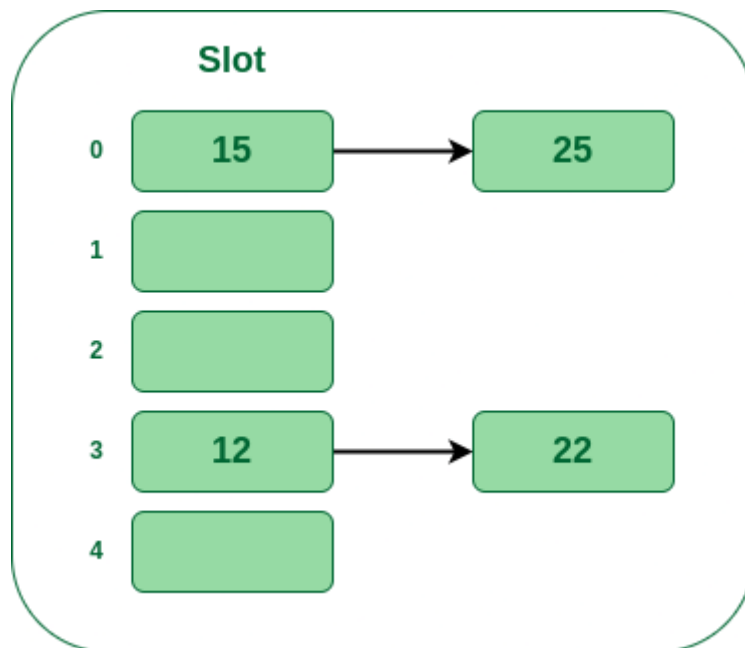
Collision in Hashing



해시 테이블에 충돌이 발생한 경우다.

분리 연결법

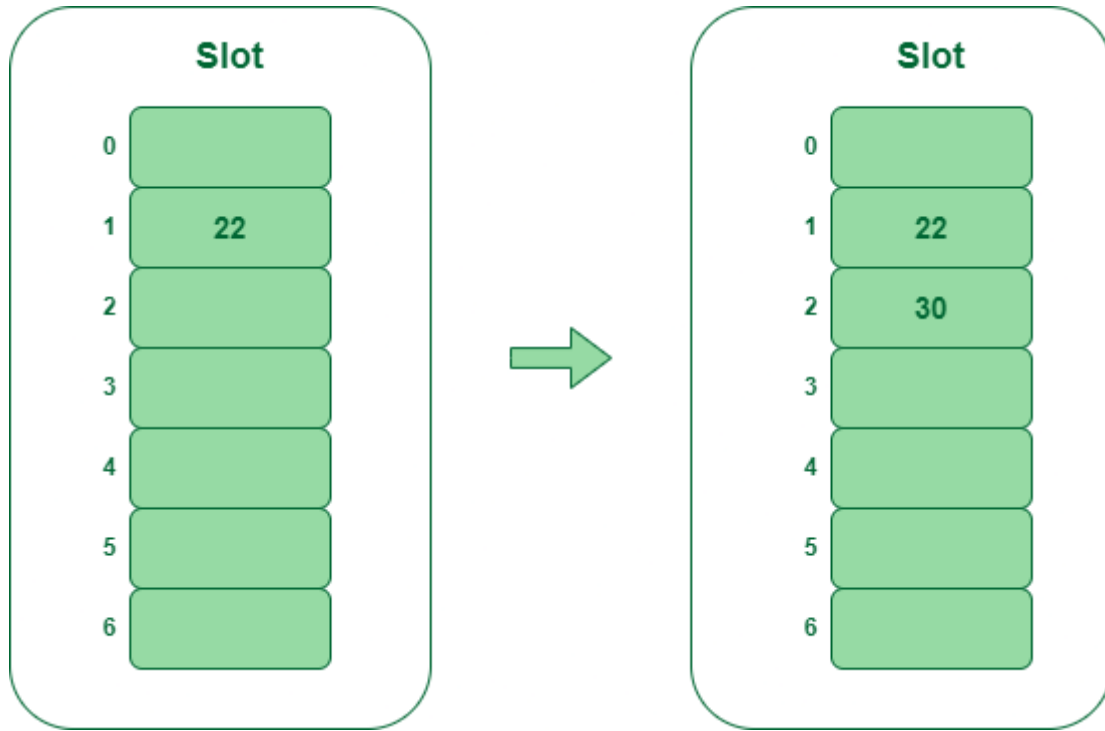
분리 연결법(Separate Chaining)은 충돌이 발생했을 때, 공간을 추가적으로 생성해 저장하는 방법이다.



체이닝을 사용하는 경우 실제 데이터는 연결 리스트에 저장된다.

개방 주소법

개방 주소법(Open Addressing)은 충돌이 발생했을 때, 해시 테이블 내 빈 공간을 찾아서 저장하는 방법이다. 여기에는 선형 조사법(Linear Probing), 2차 조사법(Quadratic Probing), 이중 해싱(Double Hashing) 등이 있다.



개방 주소법은 빈 공간을 탐색해 저장한다.

컬렉션

C#에서는 [HashSet](#)과 [Dictionary](#)로 구현이 되어 있다. 사용법은 이전에 사용해본 SortedSet과 SortedDictionary와 다를 바가 없으므로 생략한다.

더 나아가기

1. 집합(Set)과 맵(Map)의 공통점과 차이점을 데이터 저장 구조(키, 값) 관점에서 설명하시오.
2. 해시 테이블의 핵심 아이디어는 무엇이며, 왜 데이터 접근 속도가 $O(1)$ 이 될 수 있는지 배열의 특성과 연결 지어 설명하시오.
3. 해싱(Hashing)의 정의와 해시 함수(Hash Function)가 갖추어야 할 3가지 주요 성질에 대해 각각 상세히 기술하시오.
4. 해시 함수가 보안 기술(디지털 서명, 공인인증서 등)에 활용될 수 있는 이유는 해시 함수의 어떤 특징 때문인지 설명하시오.
5. 해시 테이블에서 충돌(Collision)이란 무엇이며, 메모리 구조상 왜 충돌이 필연적으로 발생할 수밖에 없는지 그 이유를 설명하시오.
6. 해시 테이블에서 충돌 해결법을 나열하고, 각각의 동작 원리를 상세히 기술해 보세요.

7. 해시 테이블의 시간 복잡도는 평균적으로 $O(1)$ 이지만, 최악의 경우(Worst Case)에는 $O(N)$ 이 될 수 있다. 어떤 상황에서 이런 성능 저하가 발생하는지 설명해 보세요.
8. 해시 테이블의 장단점은 무엇일까요?

참고자료

- [C로 배우는 쉬운 자료구조](#)
- [누구나 자료 구조와 알고리즘](#)
- [Set \(abstract data type\) - Wikipedia](#)
- [Associative array - Wikipedia](#)
- [Hashing Data Structure - GeeksforGeeks](#)
- [Hash Table \(Closed Addressing: Separate Chaining and Open Addressing: Linear Probing, Quadratic Probing, Double Hashing\) - VisuAlgo](#)
-  Hash Table은 프로그래머의 기본기
-  [보안] 해쉬와 암호화
-  이름공합과 Checksum 그리고 해시